DEMO BRIEF · OCTOBER 2026 · 5-MINUTE READ

Ask Your Data

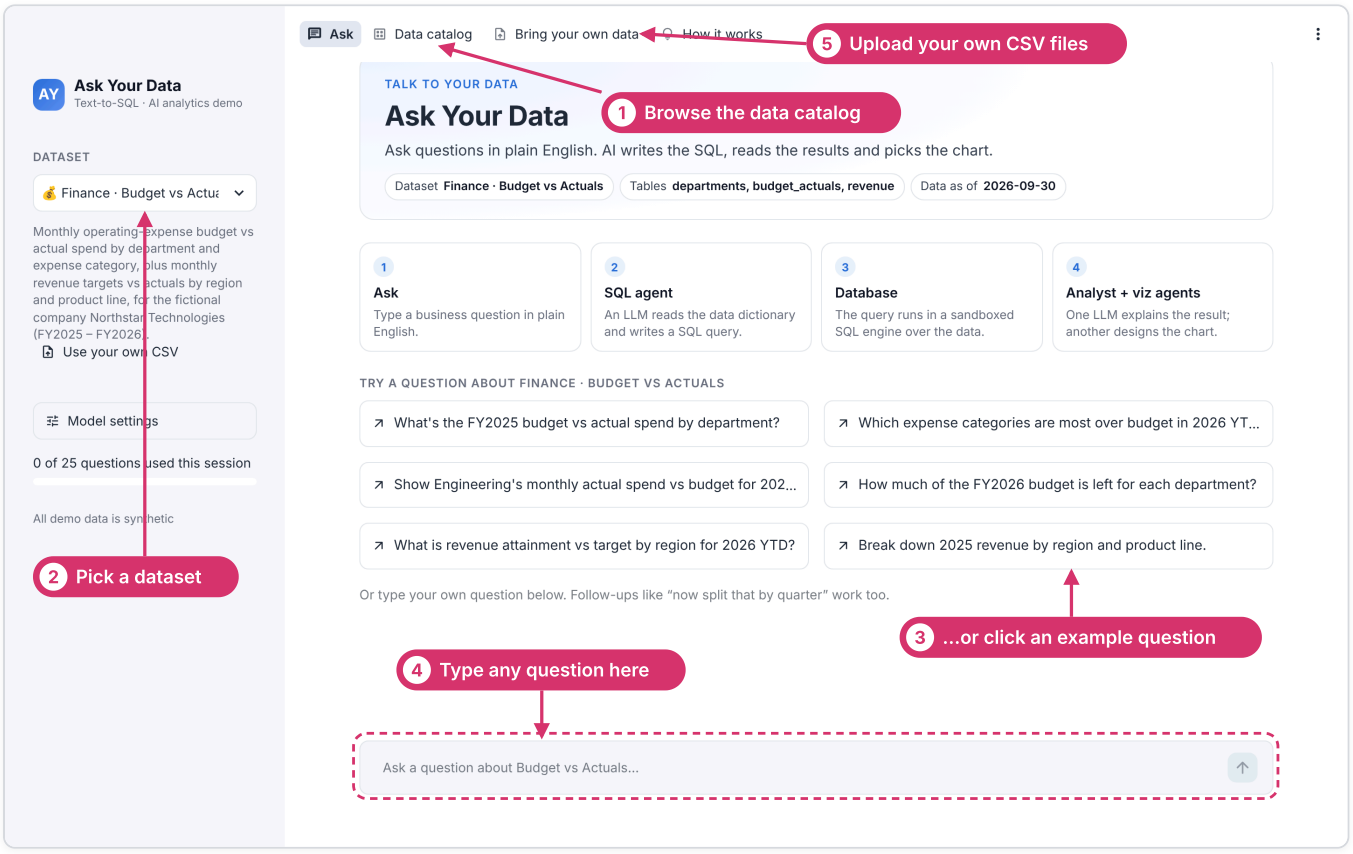
Plain-English questions over our structured data, answered with numbers, a chart and the SQL behind it.

EXECUTIVE SUMMARY

Ask Your Data lets anyone type a business question in plain English (“Which cost categories are over budget this year?”) and, within seconds, get a written answer, a chart and the exact database query used to produce it. It is a working demo on made-up data, built to answer one question: is this worth piloting on our real SAP data, one department at a time?

1 · Where you land

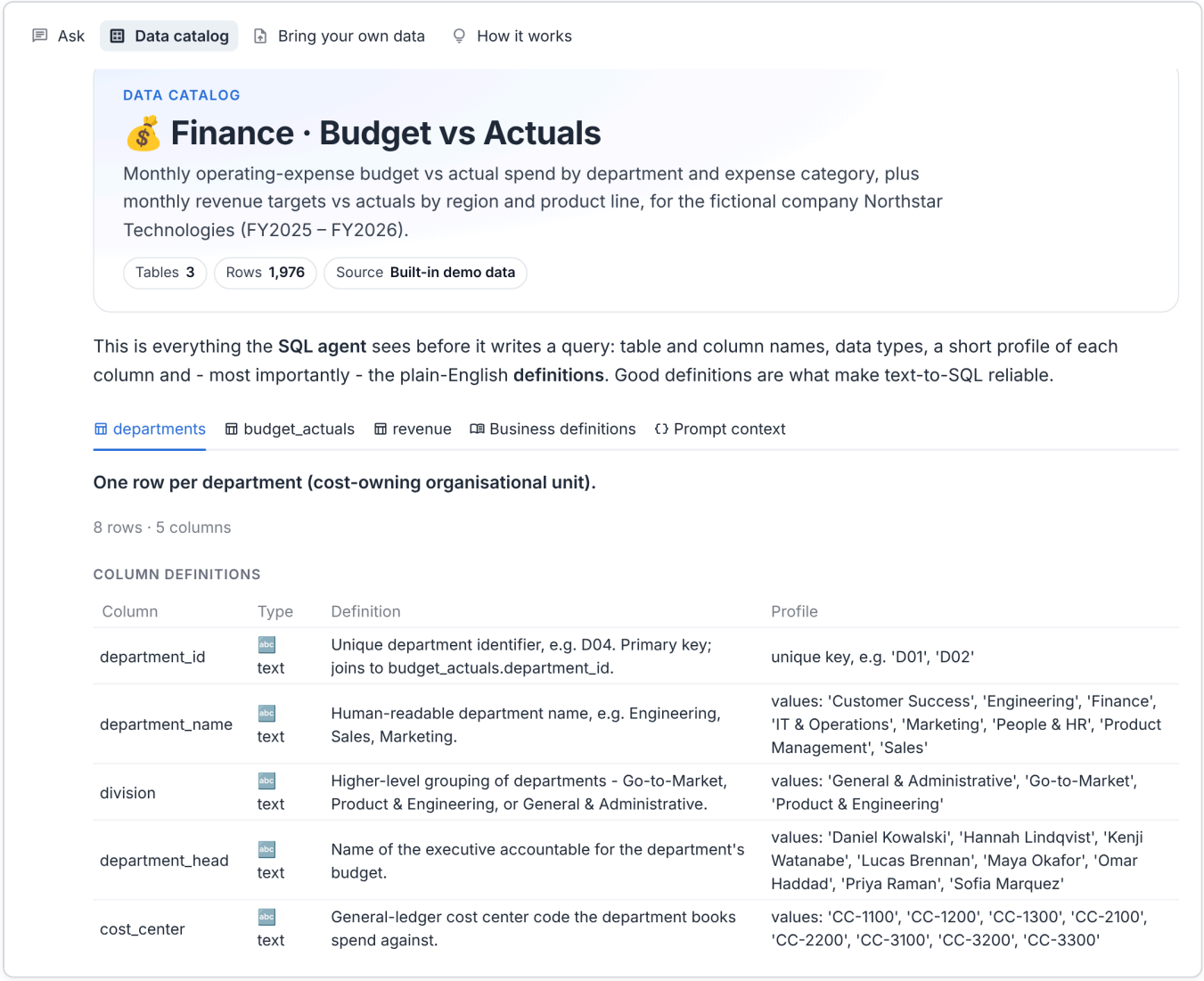
The home page. Everything a user needs is on one screen: choose a dataset, read what's in it, then ask.



1. **Data catalog**: every table and column, explained in plain English.
2. **Dataset**: Finance, HR and Projects demos (one per department in a real rollout).
3. **Example questions**: one click to see it work.
4. **Ask box**: any question; follow-ups like “now split that by quarter” work.
5. **Bring your own data**: upload a CSV and query it straight away.

2 · The data catalog (Finance example)

This is what the AI reads before it writes any query: the tables, their columns and a plain-English definition of each. The Finance demo has three tables: **departments**, **budget_actuals** (monthly budget vs. actual spend by department and cost category) and **revenue** (target vs. actual by region and product).



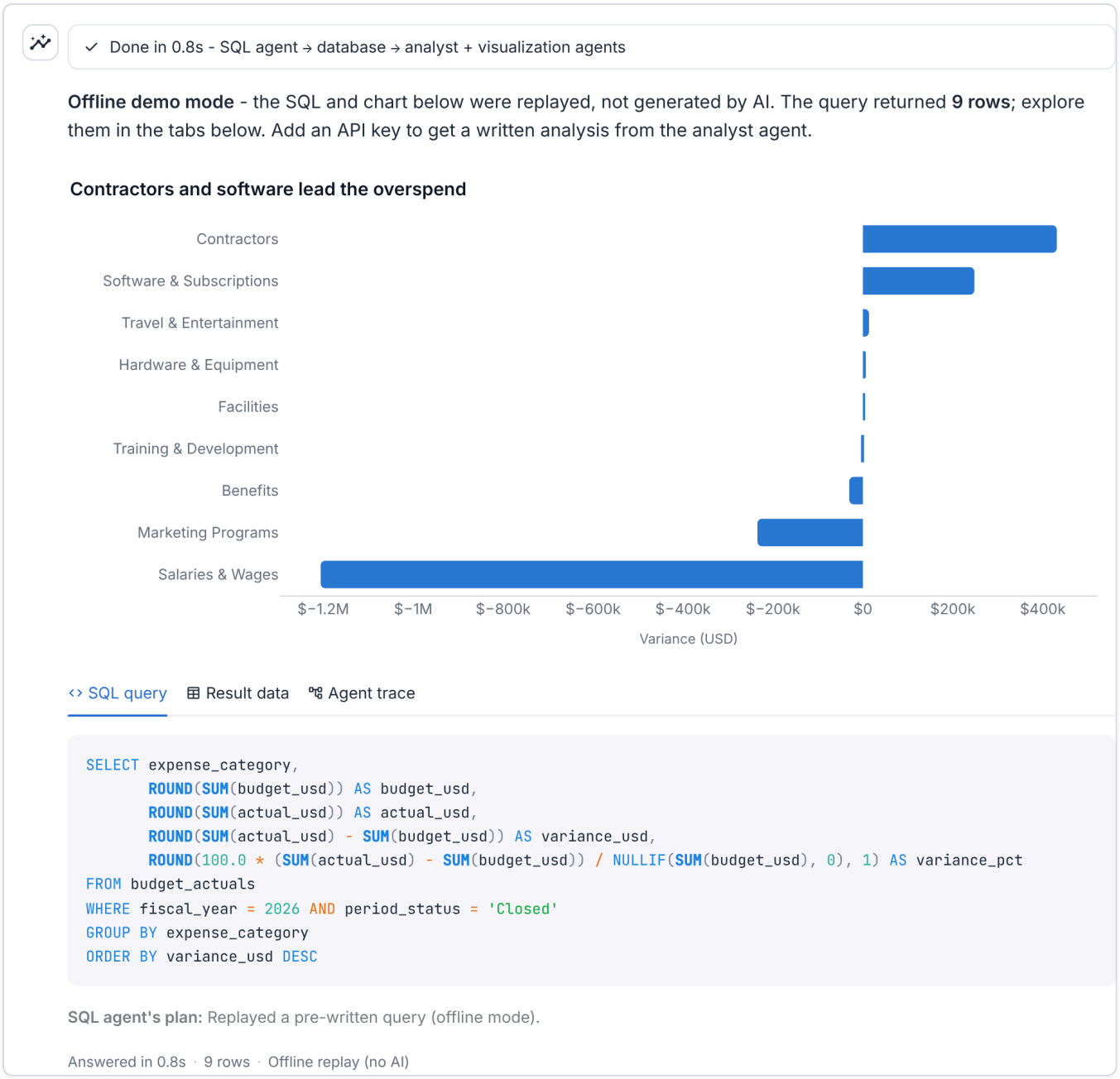
Why this matters. The quality of these definitions, plus a handful of business rules such as “YTD means closed months only”, is the single biggest lever on answer quality. In a real rollout each department would own and review its own catalog. That is analyst time, not a software project.

3 · Asking a question

QUESTION ASKED

“Which expense categories are most over budget in 2026 YTD?”

One click. The AI writes the SQL, our database runs it, then the AI explains the result in plain English and picks a chart. The **SQL query** tab under the chart shows exactly what ran, so finance can check the logic. The result rows can also be downloaded as a CSV.



Note on this screenshot: it was captured in the app's replay mode, because the AI service could not be reached from where this brief was produced. The SQL is the stored query for this sample question, but it ran live against the database, and the chart and figures are drawn from the real result. With the AI connected, the SQL is written on the fly and the analyst agent writes a short plain-English summary at the top, where the “offline demo mode” notice is now.

4 · Architecture: how the demo works

1 Question
Typed in plain English

→

2 SQL agent
AI reads the data catalog, writes one SQL query and fixes it itself if the database rejects it

→

3 Database
Query runs read-only in a locked-down SQL engine

→

4 Analyst + chart agents
One AI explains the numbers; another picks the chart, which our code draws

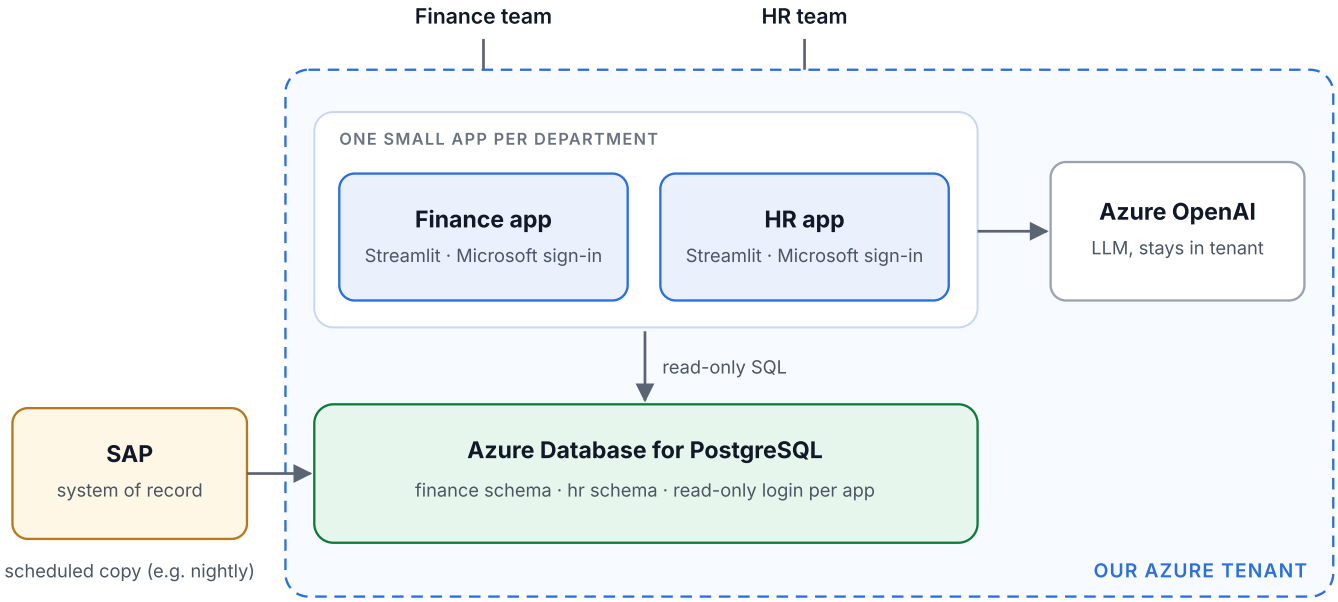
Building blocks

- **Front end:** Streamlit, a Python web-app framework. One codebase, no separate UI team needed.
- **AI:** any large language model via API (OpenAI or Anthropic today; Azure OpenAI in production).
- **Database:** today an in-memory engine (DuckDB) over CSV files of **synthetic** data. All database access goes through one module, which is the piece that changes to point at PostgreSQL.
- **Data catalog:** one simple file per dataset with table/column descriptions, business rules and example questions.
- **Packaging:** Docker image, about 100 automated tests, optional Microsoft/Google sign-in.

Built-in safety

- **Read-only.** Only a single `SELECT` is allowed. Updates, deletes and schema changes are rejected by the database's own parser.
- **No AI-written code runs.** The chart agent returns a description of the chart; our own code draws it.
- **Limits:** query timeout, row cap and a per-user question limit.
- **Transparent:** every answer shows its SQL, raw data and a step-by-step trace.
- **Data sent to the AI:** column names, a few sample values and query results. That is why production should use **Azure OpenAI inside our own tenant**.

Proposed setup on Azure (quick path)



SAP data is copied on a schedule into our Azure PostgreSQL. Each department gets its own small app that can only read its own schema. Users sign in with their Microsoft account.

5 · Two ways forward

	A · Quick path (suggested start)	B · Full build
WHAT	One Streamlit app per department (or even per person) on Azure	Python API backend + custom web front end, one platform for all departments
DATA	SAP tables copied into our Azure PostgreSQL; each app reads its own schema	Same PostgreSQL, behind a central API with a shared data catalog
ACCESS	Microsoft sign-in per app, limited to a team's Entra ID group. Everyone with access to an app sees all of that department's data	Full Active Directory integration: roles, row-level permissions, audit log
EFFORT	Small. Mostly reuses this demo; the main work is the data copy and the catalog	Large. A proper multi-month software project
TRADE-OFF	Several small apps to maintain; best for small teams	Higher cost and time; scales to the whole company

Suggested rollout: department by department

1. **Pilot with Finance.** Copy the relevant SAP tables into Azure PostgreSQL, write the Finance data catalog with a finance analyst, and deploy one app for a handful of users.
2. **Check the answers** against numbers finance already knows. Keep it only if it is accurate and saves time.
3. **Repeat for the next department** (e.g. HR) with its own data, catalog and app.
4. **Move to option B** only if several departments use it and need finer-grained permissions.

IMPORTANT

This is a demo only. It runs on synthetic data and has not been connected to SAP or any company system. It was built to test whether the idea is worth pursuing, not as a finished product.

I'd value your opinion: is this worth a small Finance pilot (option A)? Is there a department you would rather start with? Are there any concerns about sending query results to Azure OpenAI?